

Common DP Library User Guide

K-2015.06

Mar/2015

Comments?

Email your comments about this manual to:
cdpl-dev@synopsys.com.



Copyright Notice and Proprietary Information

Copyright © 2010 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Synopsys permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any. Licensee must

assign sequential numbers to all copies. These copies shall contain the following legend on the cover page:

This document is duplicated with the permission of Synopsys, Inc., for the exclusive use of _____ and its employees. This is copy number _____. ”

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AMPS, Cadabra, CATS, Synopsys Cloud Solutions, CRITIC, CSim, Design Compiler, DesignPower, DesignWare, EPIC, Formality, HSIM, HSPICE, iN-Phase, in-Sync, Leda, MAST, ModelTools, NanoSim, OpenVera, PathMill, Photolynx, Physical Compiler, PrimeTime, SiVL, SNUG, SolvNet, System Compiler, TetraMAX, VCS, Vera, and YIELDirector are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, Astro, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, Synopsys Cloud Solutions, Columbia, Columbia-CE, Cosmos, CosmosEnterprise, CosmosLE, CosmosScope, CosmosSE, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, Direct Silicon Access, Discovery, Encore, Galaxy, HANEX, HDL Compiler, Hercules, Hierarchical Optimization Technology, HSIMplus, HSPICE-Link, iN-Tandem, i-Virtual Stepper, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Magellan, Mars, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, Planet, Planet-PL, Polaris, Power Compiler, Raphael, Raphael-NES, Saturn, Scirocco, Scirocco-i, Star-RCXT, Star-SimXT, Taurus, TSUPREM-4, VCS Express, VCSi, VHDL Compiler, VirSim, and VMC are trademarks of Synopsys, Inc.

Service Marks (SM)

Common DP Library, CDPL, MAP-in, SVP Café, and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

All other product or company names may be trademarks of their respective owners.

TABLE OF CONTENTS

CDPL USAGE.....	4
INTRODUCTION	4
OPERATIONAL ENVIRONMENT	5
INFRASTRUCTURE SPECIFICATION FOR DISTRIBUTED PROCESSING.....	6
HOSTS INFORMATION FILE	6
USING THE HOST INFORMATION FILE TO SPECIFY “RSH” RUNS.....	8
USING THE HOST INFORMATION FILE TO SPECIFY “SSH” RUNS.....	8
USING THE HOST INFORMATION FILE TO SPECIFY “SGE” RUNS.....	9
USING THE HOST INFORMATION FILE TO SPECIFY “LSF” RUNS.....	9
USING THE HOST INFORMATION FILE TO SPECIFY “SH” RUNS.....	10
USING THE HOST INFORMATION FILE TO SPECIFY “PBS” RUNS.....	11
USING THE HOST INFORMATION FILE TO SPECIFY “RTDA” RUNS	11
USING THE HOST INFORMATION FILE TO SPECIFY “NETBATCH” RUNS.....	13
USING THE HOST INFORMATION FILE TO SPECIFY “CUSTOM” RUNS	14
<i>Custom submit commands.....</i>	14
<i>CUSTOM protocol.....</i>	15
API FOR HOST INFORMATION.....	16
WORKER INFORMATION FILE.....	16
COMPILE TIME CONTROL OF WORKER TYPE	17
DEBUGGING DISTRIBUTED ENVIRONMENTS	18
DPCHECK – INFRASTRUCTURE VALIDATION	18
<i>Checking a Host Information file with dpcheck.....</i>	19
<i>Checking SGE settings with dpcheck</i>	20
<i>Checking LSF settings with dpcheck</i>	20
<i>Checking PBS settings with dpcheck</i>	21
<i>Checking RTDA settings with dpcheck</i>	21
<i>Checking Netbatch settings with dpcheck</i>	21
<i>Example: An erroneous SGE configuration</i>	22
DPVALIDATE – INFRASTRUCTURE VALIDATION	22
DPANALYZER	23
LOG FILES	25
LOG VIEWING UTILITIES: LOG2TTY AND LOG2HTML.....	25
<i>Sample log from a successful run.....</i>	25
<i>Sample log with errors</i>	27
MANAGING DP LOG FILES: DPPURGE	28

1

CDPL Usage

Introduction

This document is intended for end-users of CDPL-enabled Synopsys products.

CDPL (Common Distributed Processing Library) is an internal library used by Synopsys products to enable **distributed parallel processing** of compute and data intensive applications. It provides a uniform “Master-Worker” model across multiple distributed process launch mechanisms available under Unix environment, and provides features that enable applications to offload the distributed processing functionality to the CDPL framework. This enables applications to run on multiple platforms with identical distributed processing functionality.

This document details the following end-user related concepts:

- [Hosts Information File](#)
- [Worker Information File](#)
- [LSF Interface](#)
- [SGE Interface](#)

- [PBS Interface](#)
- [RTDA Interface](#)
- [Netbatch Interface](#)

CDPL includes the DP Manager graphical User interface to access job details.

Operational Environment

When you use CDPL in your application, there are a few environment variables that are used to create and control distributed runs. These include:

CDPL_HOME: <REQUIRED>: The root for the location of CDPL-related executables and libraries.

CDPL_LOGDIR: <OPTIONAL>: The location where CDPL log files are written. If not specified, it defaults to the current working directory.

CDPL_BCASTDIR: <OPTIONAL>: The location where CDPL broadcast files are written. When unset, the default location **\$HOME/.synopsys/cdpl** is used. The broadcast files are created by the CDPL master and used by the DP Manager GUI to monitor and display information about running DP jobs.

CDPL_CUSTOM_SUBMIT: <OPTIONAL>: Set this variable to 1 to start workers with a non-standard (custom) command. The standard commands are rsh, ssh, bsub and qsub.

CDPL_CUSTOM_SUBMIT_OPTIONS: <OPTIONAL>: When using a non-standard (custom) command to start workers, this environment variable specifies a string of options to pass with the custom command. This feature is currently supported for the SSH protocol only.

Infrastructure specification for Distributed processing

For any distributed processing method using the Master-Worker flow, there is a need to specify how processes will be launched remotely by the Master application. There is also a need to indicate how many Workers can be used during a given run. These are called the “launch protocol” and “capacity” of the distributed run.

CDPL supports the following protocols for launching workers on remote nodes/machines:

- RSH [Remote Shell]
- SSH [Secure Shell]
- SGE [Oracle Grid Engine, originally Sun Grid Engine]
- LSF [Load Sharing Facility, from Platform Computing]
- SH [shell process on the local host]
- PBS [Portable Batch System]
- RTDA [Runtime Design Automation NetworkComputer]
- Netbatch [Intel Netbatch Compute Farm]

These eight methods cover most of the customer environments for distributed processing. Additional protocols are currently not supported.

It is important to note that the information specified for each of the above protocols varies.

Hosts Information File

The Hosts Information file allows the EDA tool user to specify information about the machines (and infrastructure). This information is needed for setting up the way workers are launched, as well as the capacity of workers available for the distributed run.

The hosts information file is an ASCII file. Single line comments are allowed, and must be prefixed with '#'. Every line in the file has the following format:

```
<flag>|<hostname>|<slots>|<tmpDir>|<protocol>|<command>
```

Note that all fields except the last field are delimited with '|'.

Field	Value	Purpose
Flag	Integer – can be '0' or '1'	A '0' signifies that the host cannot be used, a '1' denotes a usable WORKER machine
Hostname	String, valid hostname	This is the hostname of a valid WORKER machine. For SGE , LSF, PBS, RTDA, Netbatch and CUSTOM, this field is empty. For SH, this field is ignored.
Slots	Integer – any valid integer	This number is used to denote the number of worker slots available on the host (for RSH, SSH and SH) or in the farm (for LSF, SGE, PBS, RTDA, Netbatch and CUSTOM). -1 denotes "unlimited". When using -1, there is no hard limit on the number of workers on the host. In this case, CDPL will create as many workers as it needs for a job up to the maximum configured for the worker type.
Temporary directory	String – a valid, writable directory	This directory is used for local data storage for an application – if needed. A worker can get the temporary directory name from the CDPL_TMPDIR environment variable.
Protocol	String identifier	This string id signifies the command-mode used to connect to the WORKER machine. Valid IDs are RSH, SSH, SH, SGE, LSF, PBS, RTDA, NB and CUSTOM.
Command	String – the connection command	The given string denotes the actual command string used to connect to the WORKER machine. For example, a valid combination is "RSH" for field 5 and 'rsh' for field 6. For LSF, SGE, PBS, RTDA and NB, you may need to provide command parameters also.

Using the Host Information File to specify “RSH” runs

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots| tmpDir| protocol | command
#
1|teralab-x15| 4| /remote/users/tmp| RSH| rsh
1|teralab-x16| 8| /remote/users/tmp| RSH| rsh
```

This information file specifies a 12 worker, 2 node configuration for a distributed processing run. Note that these are the maximum workers that will be launched. CDPL will launch up to 4 workers on teralab-x15 and up to 8 workers on teralab-x16.

Using the Host Information File to specify “SSH” runs

```
# HOSTS INFORMATION FILE (SSH)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots| tmpDir| protocol | command
#
1|cdpl_lab-x15| 4| /remote/users/tmp| SSH| ssh
1|cdpl_lab-x16| 8| /remote/users/tmp| SSH| ssh
```

The above information file specifies a 12 worker, 2 machine SSH infrastructure.

CDPL with SSH requires password-less login. The following steps set up password-less login for SSH.

```
% cd ~/.ssh
% ssh-keygen -f id_rsa -N '' -t rsa -q
[You do not need a pass phrase.]
% cp id_rsa.pub authorized_keys
```

Using the Host Information File to specify “SGE” runs

SGE (Sun Grid Engine) runs can be customized by using SGE-specific Hosts Information files. For SGE, the following format is used:

```
# HOSTS INFORMATION FILE (SGE)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting SGE jobs
1|  |3|/remote/users/tmp|SGE| qsub -P bnormal -l arch=glinux
```

Note that the 'hostname' field is left empty for SGE.

This example specifies 3 worker slots on Linux target systems.

To find out if your SGE installation is valid, and ready to be used, use the [dpcheck](#) utility script.

Using the Host Information File to specify “LSF” runs

LSF runs can be customized by using LSF-specific Hosts Information file. For LSF, the following format is used:

```
# HOSTS INFORMATION FILE (LSF)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting LSF jobs
1| -1|/remote/users/tmp|LSF| bsub -P bnormal
```

Note that the 'hostname' field is left empty for LSF. This example specifies an unlimited number of worker slots.

To specify a required number of slots for a given LSF run, the following format may be used:

```
# HOSTS INFORMATION FILE (LSF)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol#| command
# The end-user environment should be properly configured and capable of
# starting LSF jobs
1| 3|/remote/users/tmp|LSF| bsub -q bnormal
```

This example specifies a 3 slot farm capacity for using LSF. Only 3 workers will be launched by CDPL in this instance.

Using the Host Information File to specify “SH” runs

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol | command
#
1|localhost| 8| /remote/users/tmp| SH| sh
```

This information file specifies up to 8 workers on the localhost. When using the SH protocol, the hostname field is not required to be set because all the workers will be run on the local host. Here, we have set hostname to “localhost” as a reminder.

The SH protocol can be used when rsh and ssh are not available, but you still want your application to use CDPL to distribute the processing over multiple cores on the same machine.

Using the Host Information File to specify “PBS” runs

PBS (Portable Batch Systems) runs can be customized by using PBS-specific Hosts Information files. For PBS, the following format is used:

```
# HOSTS INFORMATION FILE (PBS)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting PBS jobs
1|  |16|/remote/users/tmp|PBS| qsub -q bnormal -l arch=linux
```

Note that the 'hostname' field is left empty for PBS.

This example specifies 16 worker slots on Linux target systems.

To find out if your PBS installation is valid, and ready to be used, use the [dpcheck](#) utility script.

Using the Host Information File to specify “RTDA” runs

RTDA (Runtime Design Automation) runs can be customized by using RTDA-specific Hosts Information files. For RTDA, the following format is used:

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1| |10|/remote/users/tmp| RTDA | nc run
```

Note that the 'hostname' field is left empty for RTDA. This example specifies 10 worker slots.

Valid RTDA command parameters can follow the “**nc run**” command string.

To find out if your RTDA installation is valid, and ready to be used, use the [dpcheck](#) utility script.

The RTDA **nc** command may conflict with the Linux **/usr/bin/nc** command in your user environment. If necessary, you can specify the full path to the RTDA **nc** command in your RTDA configuration file.

The following example is similar to the one above except that it specifies the full path to the RTDA **nc** command.

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1| |10|/remote/users/tmp| RTDA |/remote/fsd/rtda/2011.03/linux/scripts/nc run
```

Full Path to nc Command

If you submit the application master to the RTDA farm, and configure it to start RTDA workers, you must add the **-e “SNAPSHOT”** option to your **nc run** command in order to start the workers with the correct environment settings.

The following example shows a configuration where the RTDA workers are started with the **-e “SNAPSHOT”** option.

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots| tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1| | 10| /remote/users/tmp| RTDA | nc run -e "SNAPSHOT"
```

Command with Environment Snapshot Option

Using the Host Information File to specify “Netbatch” runs

Intel Netbatch runs can be customized by using Netbatch-specific Hosts Information files. For Netbatch, the following format is used:

```
# HOSTS INFORMATION FILE (Netbatch)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots| tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting Netbatch jobs
1| | 8 | /remote/users/tmp | NB | nbjob run
```

The protocol field for a Netbatch host information file is “**NB**”.

Note that the ‘hostname’ field is left empty for Netbatch. This example specifies 8 worker slots.

Valid Netbatch command parameters can follow the “**nbjob run**” command string.

To find out if your Netbatch installation is valid, and ready to be used, use the [dpcheck](#) utility script.

Using the Host Information File to specify “Custom” runs

CDPL provides support for custom submit commands and the CUSTOM protocol to work with customers who have established methods of submitting DP jobs and are unable to directly use the CDPL submit commands or protocols.

Custom submit commands

Some sites have existing DP submission methods already defined and are not able to use one of the CDPL standard commands `rsh`, `ssh`, `bsub`, `qsub`, `sh`, `nc` or `njob`. In this case, you can configure CDPL to use a custom submit command to start the workers.

Use of the custom submit command has the following restrictions.

1. The command must use one of the protocols CDPL supports, e.g. `RSH`, `SSH`, `SH`, `LSF`, `SGE`, `PBS`, `RTDA` or `Netbatch`.
2. The custom command must be command line compatible with the original command.
3. For `LSF` and `SGE`, the command must support the way CDPL submits the worker job which is via standard input.

The following steps set up a custom worker submit command.

1. Set the environment variable `CDPL_CUSTOM_SUBMIT` to 1 for the CDPL master.
2. Specify your custom command in the Command field of the Hosts Information file.

In the hosts file, you will specify the protocol that matches your submit method. In the following example, we show a Hosts Information File that uses a custom submit command similar to `rsh`.

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots| tmpDir| protocol | command
#
# Use /usr/local/bin/custom_cmd to start the workers.
1|teralab-x15| 4| /remote/users/tmp| RSH| /usr/local/bin/custom_cmd
1|teralab-x16| 8| /remote/users/tmp| RSH| /usr/local/bin/custom_cmd
```

CUSTOM protocol

The CUSTOM protocol is more flexible than the custom submit command in configuring CDPL for sites that have in-house DP submission commands defined for their compute farms. CUSTOM protocol does not have any of the three restrictions necessary for using a custom submit command. When using CUSTOM protocol, CDPL submits the job by prepending the user-defined submit command to the worker command.

CUSTOM protocol can be used for sites that prefer their own wrapper scripts to submit jobs to their compute farm. It can be used for supported compute farms like LSF and SGE, and even for compute farms that CDPL does not directly support.

The following example shows a Hosts Information File that uses CUSTOM protocol to submit the workers to a compute farm.

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots| tmpDir| protocol | command
#
# Use CUSTOM protocol to start the workers.
1| | 4| /remote/users/tmp| CUSTOM | myfarm_submit_command <options>
```

API for Host Information

All of the above methods are configurable at runtime, and suit most applications. For advanced applications which need a fine degree of control over their capacity, CDPL provides a method of providing the host data using the **cdplAddHost** call during Master initialization instead of using the host information file.

Worker Information File

The Worker Information file allows the EDA tool user to specify information about the distributed processing workers. This information includes the following.

- The type of worker required by the Master. This is an integer number and typically is mutually exclusive for multiple applications running under CDPL. If applications wish to ensure that these numbers do not conflict when multiple applications are used in a flow, each worker type has to be a UNIQUE integer, and care must be taken by the CAD user that this happens.
- The WORKER executable name and location
- The environment and configuration to be used by the WORKER

The Worker Information file is an ASCII file. Single line comments are allowed, and must be prefixed with '#'. Every line in the file has the following format.

<worker_type>; <max_workers>; <min_workers>; <worker_executable and parameters>

Field	Value	Purpose
Worker type	Integer	The type of worker for executing the application master tasks. This number is pre-negotiated with the CDPL team.
Max workers	Integer	The maximum number of workers. CDPL attempts to start as many workers it can, up to the maximum specified in this file.
Min workers	Integer	The minimum number of WORKERS needed for successful operation. It should be at least '1'.
Worker executable	String – a valid executable program or script	The full path of the worker executable. The worker executable can be prepended with a wrapper script that can be very useful in providing visibility into worker operation. A sample wrapper script (START_WORKER) is provided in the tutorials subdirectory of the CDPL SDK.
parameters	String	Application specific parameters the worker executable will need. The worker executable and parameters form the command line that will be executed.

Here is a sample Worker Information file.

```
# Worker Information file
#
10; 1; 1; /home/user1/cdpl/tutorial/START_WORKER /home/user1/cdpl/tutorial/countcomments -worker
```

Compile time control of worker type

Note that an alternate method of providing the Worker Information data is to use the [cdplAddWorker](#) call during Master initialization instead of using the Worker Information file.

If Applications need to exert control over the worker type integer, they can programmatically negotiate a random integer for each worker binary that they plan to use within their application. CDPL provides an API call ([cdplAddWorkerAuto](#)) to create unique worker types within the application at compile time.

2

Debugging Distributed Environments

Quite often, distributed processing environments depend on customer configuration of their infrastructure. This can be very difficult to predict in advance, and some of these configurations may not be part of Synopsys CDPL's native environment configuration.

To aid field engineers and R&D in such cases, CDPL provides a few methods to analyze and validate the DP environment, as well as provide tools or utilities to capture detailed run information.

DPCHECK – Infrastructure Validation

`dpcheck` is a utility distributed with the CDPL source code and SDK. The utility script can be used to ensure that the SGE/LSF/PBS/RTDA/NB infrastructure is ready to accept jobs. This script is available at `$CDPL_HOME/bin`. Run `dpcheck -h` to get the correct usage.

Here is the output from 'dpcheck -h'.

```
% dpcheck -h
```

Usage:

```
dpcheck -h|--help
```

```
dpcheck -m {rsh|sge|lsf|ssh|sh|pbs|rtda|nb}
```

```
dpcheck -host {host file}
```

CDPL needs rsh|ssh|lsf|sge|sh|pbs|rtda|nb to be set correctly for launching workers.

This program allows you to validate the transport settings.

The following checks are performed with the options below.

-m:

rsh|ssh: check if localhost login is allowed (passwordless)

sge|lsf|pbs|rtda|nb: check for environment settings, submit host and default queues

automount check on localhost.

-host:

This checks the availability of the hosts listed in the host file.

Also, all the checks done with the -m option are performed here.

An exit code of 0 signifies the proper environment.

Checking a Host Information file with dpcheck

To check if your host information file configuration, use the command 'dpcheck -host <hostfile>'. In the following example, dpcheck is used to check the configuration of an rsh host information file.

```
% dpcheck -host hosts.rsh
automount process is running at localhost.
Valid hostfile found for DP run.
Valid [rsh] environment found for DP runs.
    : t3amd03: Fri Nov  5 10:20:33 PDT 2010
    : t3amd01 is alive
    : t3amd02 is alive
    : t3amd03 is alive
    : t3amd01: Fri Nov  5 10:20:33 PDT 2010
    : t3amd02: Fri Nov  5 10:20:34 PDT 2010
    : t3amd03: Fri Nov  5 10:20:34 PDT 2010
```

Checking SGE settings with dpcheck

To check if SGE settings are correctly configured for a given user id, login to your interactive shell and type:

```
% dpcheck -m sge

Valid [sge] environment found for DP runs.
    : teralab-x4150 is a valid submit host
```

Checking LSF settings with dpcheck

Checking LSF environment is similar to SGE: run the following command:

```
% dpcheck -m lsf

Valid [lsf] environment found for DP runs.
    : Default queue is [bnormal]
```

Checking PBS settings with dpcheck

Checking the PBS environment is similar to LSF and SGE.

```
% dpcheck -m pbs
```

```
Valid [pbs] environment found for DP runs.
```

```
  : t3syn07 is a valid submit host
```

Checking RTDA settings with dpcheck

Checking the RTDA environment is similar to LSF and SGE.

```
% dpcheck -m rtda
```

```
Valid [rtda] environment found for DP runs.
```

```
  : NC_QUEUE is vnclogic
```

```
  : Submit command: /remote/fsd/rtda/2011.03/linux/scripts/nc submit
```

```
  : t3syn03 is a valid submit host
```

Checking Netbatch settings with dpcheck

Checking the Netbatch environment is similar to LSF and SGE.

```
% dpcheck -m nb
```

```
Valid [nb] environment found for DP runs.
```

Many simple errors can be detected using this utility.

Example: An erroneous SGE configuration

Here is an example of a machine that is not configured to run SGE on a given farm.

```
% dpcheck -m sge
```

Errors in [sge] check:

**error running /remote/sge/default/bin/lx24-amd64/qconf: error:
denied: host "snpsamdNNN" is neither submit nor admin host**

DPVALIDATE – Infrastructure Validation

“**dpvalidate**” is a utility distributed with the CDPL source code and SDK. This utility script can be used to check the readiness of the environment for CDPL. This script is available at \$CDPL_HOME/bin. Run ‘dpvalidate’ to get the correct usage.

Here is the usage of ‘dpvalidate’.

```
% dpvalidate
```

```
Setting launch timeout to 30 seconds!
```

```
Usage: dpvalidate [-c -p port -h host [-m message]]
```

```
dpvalidate -l {rsh|ssh|qsub|bsub|sh}
```

“dpvalidate” can tell you if your LSF or SGE farm environment is ready to use CDPL. This is because it simulates a CDPL connection to the farm using the default bsub (LSF), or qsub (SGE) command.

Here is an example of the output of ‘dpvalidate’ when checking an LSF farm.

```
% dpvalidate -l bsub
```

```
Setting launch timeout to 30 seconds!  
Platform LSF HPC 7 Update 6, Sep 04 2009  
Copyright 1992-2009 Platform Computing Corporation
```

```
My cluster name is cdpl  
My master name is t3master01  
CDPL safe!
```

DPMasterHost	DPClient	Slot	Return_Code
t3sys03	t3sys01 1	9080	SUCCESS [bsub], bsub

The next example shows the output of 'dpvalidate' when checking an SGE farm.

```
% dpvalidate -l qsub
```

```
Setting launch timeout to 30 seconds!  
CDPL safe!
```

DPMasterHost	DPClient	Slot	Return_Code
t3sys03	t3sys06 1	9080	SUCCESS [qsub], qsub

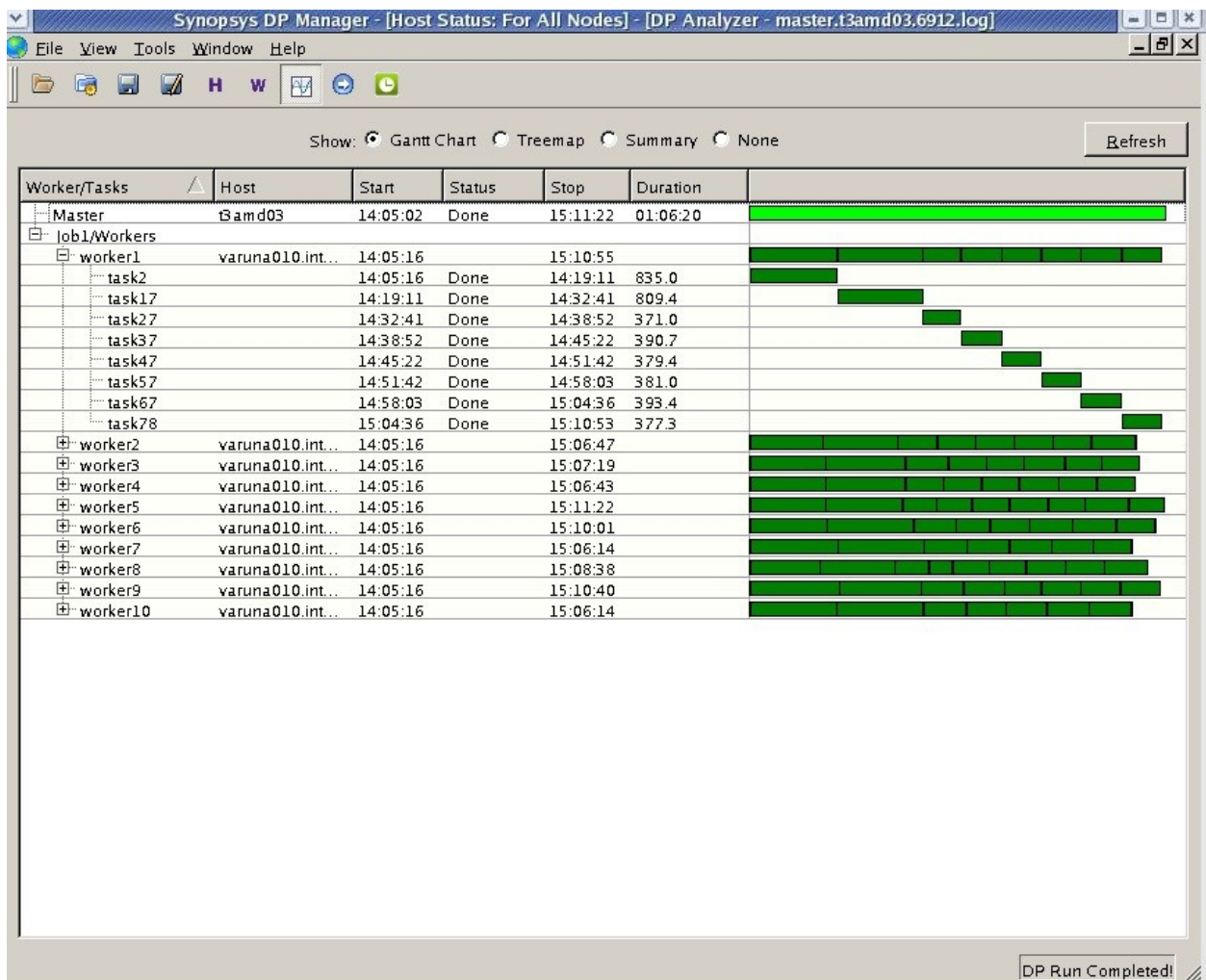
If 'dpvalidate' reports errors when trying to connect to your LSF, SGE, PBS or RTDA farm, you will need to identify the site-specific requirements of the farm and integrate CDPL with them. This may involve integrating CDPL with site-specific job submission commands or security methods.

DPANALYZER

DPAnalyzer is a widget provided within the CDPL DP Manager GUI as part of our SDK release. It is a graphical screen that allows you to visually inspect a

running or finished job, and analyze certain characteristics of the job and tasks.

The display below shows the DP Analyzer Gantt chart view of a completed job. One noteworthy characteristic in this view is the white space in each worker's task timeline. This indicates time during which the worker was not actually executing a task assigned by the CDPL master. When this type of behavior is unexpected, further analysis should be done to determine why the worker is not actively executing a task during this time.



Log Files

A typical CDPL application will generate several log files in addition to the required application logs. These logs are informative and useful when investigating problems with a DP run.

The following log files will be created during a DP run.

```
master.<hostname>.<pid>.log  
worker.W1.<hostname>.<pid>.log  
...  
worker.Wn.<hostname>.<pid>.log
```

The number of worker log files will be equal to the number of workers that ran during the job.

The master log file provides the most information about the DP run. Always check the master log first when investigating problems.

Log Viewing Utilities: log2tty and log2html

These utilities are found in the \$CDPL_HOME/bin directory. They filter output from the master and worker log files into a more readable format. Use log2tty when viewing the output in a command shell window. Use log2html to generate output for viewing in a web browser. The typical usage is the following.

log2tty <filename>

log2html <filename> > <filename.html>

where <filename> is a master or worker log file.

Sample log from a successful run

Here is an example of the output from log2tty when viewing a master log file from a successful DP run. This DP run is a small job with 3 workers and 3 tasks.

```
sh-3.00$
sh-3.00$ log2tty master.t3amd03.11985.log
PDT 2010/10/28 11:32:17.500484 -- time: 2010/10/28 11:32:17 PDT -0700
PDT 2010/10/28 11:32:17.500491 -- version: CDPL_E2010.09_SP1_PROD_BF2 1616494
PDT 2010/10/28 11:32:17.500496 -- system: Linux 2.6.9-67.ELsmp #1 SMP Wed Nov 7 13:56:44 EST 200
7
PDT 2010/10/28 11:32:17.500500 -- processor: x86_64 64-bits
PDT 2010/10/28 11:32:17.500504 -- hostname: t3amd03
PDT 2010/10/28 11:32:17.500526 -- new server S975361824
PDT 2010/10/28 11:32:17.501058 -- new master M1
PDT 2010/10/28 11:32:17.501067 -- reading hostfile `etc/hosts'
PDT 2010/10/28 11:32:17.516627 -- creating RSH farm
PDT 2010/10/28 11:32:17.516640 -- exporting CDPL_LOGDIR as `/remote/t3dev1/geiszler/cdpl/compat/
samples/cdplgui'
PDT 2010/10/28 11:32:17.516644 -- exporting CDPL_TMPDIR as `/tmp'
PDT 2010/10/28 11:32:17.516649 -- exporting CDPL_HOME as `/remote/t3dev1/geiszler/cdpl'
PDT 2010/10/28 11:32:17.516652 -- exporting DISPLAY as `:1.0'
PDT 2010/10/28 11:32:17.516805 -- bcast dir is `/remote/us01home21/geiszler/.synopsys/cdpl'
PDT 2010/10/28 11:32:17.542220 -- new job J1 in M1
PDT 2010/10/28 11:32:17.542232 -- new worker type 10, min:1, max:4, cmd:`/remote/t3dev1/geiszler
/cdpl/compat/samples/cdplgui/countcomments -worker'
PDT 2010/10/28 11:32:17.542238 -- new bin B1 in J1
PDT 2010/10/28 11:32:17.542252 -- new task T0 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542259 -- new task T1 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542264 -- new task T2 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542283 -- starting job J1
PDT 2010/10/28 11:32:17.542365 -- started worker W1 (as C1) in bin B1
PDT 2010/10/28 11:32:17.542383 -- started worker W2 (as C2) in bin B1
PDT 2010/10/28 11:32:17.542400 -- started worker W3 (as C3) in bin B1
PDT 2010/10/28 11:32:33.053556 -- W3 is `t3amd03:57859'
PDT 2010/10/28 11:32:33.053592 -- assigning task T0 to worker W3
PDT 2010/10/28 11:32:33.109431 -- worker W3 completed task T0
PDT 2010/10/28 11:32:33.109435 -- stat T0: utime=1 stime=1
PDT 2010/10/28 11:32:33.109440 -- statm T0: vsize=29232 rss=944 shared=772 text=272 data=20792
PDT 2010/10/28 11:32:33.109469 -- assigning task T1 to worker W3
PDT 2010/10/28 11:32:33.116319 -- W2 is `t3amd02:40594'
PDT 2010/10/28 11:32:33.116343 -- assigning task T2 to worker W2
PDT 2010/10/28 11:32:33.141454 -- W1 is `t3amd01:50858'
PDT 2010/10/28 11:32:33.170510 -- worker W2 completed task T2
PDT 2010/10/28 11:32:33.170514 -- stat T2: utime=0 stime=2
PDT 2010/10/28 11:32:33.170518 -- statm T2: vsize=29232 rss=944 shared=772 text=272 data=20792
PDT 2010/10/28 11:32:33.208484 -- worker W3 completed task T1
PDT 2010/10/28 11:32:33.208488 -- stat T1: utime=0 stime=0
PDT 2010/10/28 11:32:33.208492 -- statm T1: vsize=29232 rss=964 shared=792 text=272 data=20792
PDT 2010/10/28 11:32:33.558410 -- freeing job J1
PDT 2010/10/28 11:32:33.558428 -- stopping job J1
PDT 2010/10/28 11:32:33.561830 -- freeing server S975361824
PDT 2010/10/28 11:32:33.562325 -- killing server
PDT 2010/10/28 11:32:33.562422 -- freeing master M1
PDT 2010/10/28 11:32:33.562675 -- freeing farm
sh-3.00$
```

Sample log with errors

Here is an example of the log output from log2tty from a DP run that had errors and did not complete. In this example, the error occurred because the worker executable failed to run on the remote system. The first thing to check with an error like this is whether the worker executable file is available on the remote system and can be run there.

```

sh-3.00$
sh-3.00$ log2tty master.t3amd03.24312.log
PDT 2010/10/28 11:50:00.271711 -- time: 2010/10/28 11:50:00 PDT -0700
PDT 2010/10/28 11:50:00.271719 -- version: CDPL_E2010.09_SP1_PROD_BF2 1616494
PDT 2010/10/28 11:50:00.271723 -- system: Linux 2.6.9-67.ELsmp #1 SMP Wed Nov 7 13:56:44 EST 200
7
PDT 2010/10/28 11:50:00.271730 -- processor: x86_64 64-bits
PDT 2010/10/28 11:50:00.271733 -- hostname: t3amd03
PDT 2010/10/28 11:50:00.271755 -- new server S1534418917
PDT 2010/10/28 11:50:00.272440 -- new master M1
PDT 2010/10/28 11:50:00.272449 -- reading hostfile `hosts'
PDT 2010/10/28 11:50:00.301588 -- creating RSH farm
PDT 2010/10/28 11:50:00.301615 -- exporting CDPL_LOGDIR as `/remote/t3dev1/geiszler/cdpl/compat/
samples/cdplgui'
PDT 2010/10/28 11:50:00.301619 -- exporting CDPL_TMPDIR as `${CDPL_HOME}'
PDT 2010/10/28 11:50:00.301625 -- exporting CDPL_HOME as `/remote/t3dev1/geiszler/cdpl'
PDT 2010/10/28 11:50:00.301629 -- exporting DISPLAY as `:1.0'
PDT 2010/10/28 11:50:00.301721 -- bcast dir is `/remote/us01home21/geiszler/.synopsys/cdpl'
PDT 2010/10/28 11:50:00.304843 -- new job J1 in M1
PDT 2010/10/28 11:50:00.304865 -- new worker type 10, min:1, max:4, cmd:`/remote/t3dev1/geiszler
/cdpl/compat/samples/cdplgui/countcomments -worker'
PDT 2010/10/28 11:50:00.304871 -- new bin B1 in J1
PDT 2010/10/28 11:50:00.304903 -- new task T0 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304911 -- new task T1 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304921 -- new task T2 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304940 -- starting job J1
PDT 2010/10/28 11:50:00.305041 -- started worker W1 (as C1) in bin B1
PDT 2010/10/28 11:50:00.305058 -- started worker W2 (as C2) in bin B1
PDT 2010/10/28 11:50:00.305074 -- started worker W3 (as C3) in bin B1
PDT 2010/10/28 11:55:00.354184 -- W1 has timed out
PDT 2010/10/28 11:55:00.354244 -- W2 has timed out
PDT 2010/10/28 11:55:00.354271 -- W3 has timed out
PDT 2010/10/28 11:55:00.354294 -- 3 workers have timed out in B1
PDT 2010/10/28 11:55:00.354307 -- worker command for B1 appears to be invalid
PDT 2010/10/28 11:55:00.354314 -- offending command is `/remote/t3dev1/geiszler/cdpl/compat/samp
les/cdplgui/countcomments -worker'
PDT 2010/10/28 11:55:00.354320 -- B1 bailing out
PDT 2010/10/28 11:55:00.354334 -- killing job J1
PDT 2010/10/28 11:55:00.615138 -- freeing job J1
PDT 2010/10/28 11:55:00.618111 -- freeing server S1534418917
PDT 2010/10/28 11:55:00.618447 -- killing server
PDT 2010/10/28 11:55:00.618533 -- freeing master M1
PDT 2010/10/28 11:55:00.619349 -- freeing farm
sh-3.00$

```

Managing DP Log Files: dppurge

The DP log files will accumulate over time. Unless they are periodically deleted, they could eventually have a negative effect on system performance. The files that will accumulate over many DP runs are the master and worker log files, and also the broadcast files in the **\$HOME/.synopsys/cdpl** directory.

CDPL provides the dppurge utility to find and remove files from old DP jobs. dppurge is found in the \$CDPL_HOME/bin directory. Here is the dppurge command line usage.

dppurge -h

Usage:

```
dppurge [-k numjobs] [-b bcastdir]
```

```
dppurge -h|--help
```

This program removes files from old DP jobs.
The files removed are the broadcast files, master logs and worker logs.
The following options are available.

-k numjobs:

specifies the number of jobs to keep.

By default, files from all jobs are removed. This is equivalent to “-k 0”

-b bcastdir:

Specifies the broadcast directory used to locate the DP files.